

## Ce que vous aurez à la fin

Le modèle de données complet du suivi des stages — entreprises, offres, compétences, étudiants, candidatures, stages, tuteurs et enseignants référents —, découpé en un fichier par modèle, administrable, et peuplé d'un jeu de données cohérent que vous saurez interroger depuis le shell.

Aucun modèle n'est donné. Vous le tirez de ce que dit la responsable des stages, et vous défendez chacun de vos choix.

## Point de départ

Repartez de votre dépôt de la séance Mise en route. Si vous l'avez manquée, ou si votre projet est parti de travers, repartez du dépôt du cours :

```
git clone https://github.com/vianney-g/ifnti_django_projet.git
cd ifnti_django_projet
git switch -c tp2 tp2-depart
uv sync
```

## 1. Un modèle par fichier (15 min)

Transformez `stages/models.py` en un package `stages/models/`, avec un fichier `entreprise.py` pour le modèle existant et un `__init__.py` qui le réexporte. Rien d'autre ne doit changer : l'admin, la vue et les tests doivent continuer de fonctionner sans modification.

Lancez ensuite `uv run manage.py makemigrations`.

a. Que répond Django ? Vous avez pourtant déplacé et renommé le fichier du modèle : qu'est-ce que cela vous apprend sur la façon dont Django identifie un modèle, et sur ce qu'est une migration ?

Commitez avant d'aller plus loin : ce déplacement mérite un commit à lui seul.

## 2. La parole de la responsable des stages (55 min)

Voici ce qu'elle vous a expliqué, dans ses mots.

« Chaque année, les étudiants de L3 font un stage de fin d'études. Les entreprises nous envoient des offres : un intitulé, une description, les dates de début et de fin, le nombre de stagiaires qu'elles prennent, et les compétences qu'elles attendent.

Les étudiants candidatent aux offres qui les intéressent — autant qu'ils veulent, mais jamais deux fois à la même. On suit chaque candidature : déposée, puis retenue ou refusée. Quand une candidature est retenue, elle devient un stage. L'entreprise désigne alors un tuteur parmi ses employés, et l'IFNTI un enseignant référent. Le stage a un sujet, qui n'est pas forcément l'intitulé de l'offre.

Pour chaque personne — étudiant, tuteur ou enseignant —, on garde le nom, le prénom, le sexe, la date de naissance et l'adresse mail. Les étudiants ont en plus un matricule et une promotion, et on note leurs compétences, pour pouvoir les rapprocher des offres.

Et surtout : on ne perd jamais l'historique d'un stage. Même si une entreprise ferme, même si un étudiant a quitté l'IFNTI depuis des années, on doit pouvoir retrouver qui a fait quel stage, où, et avec qui. »

## 2.1 Les noms sont imposés, pas les décisions

La semaine prochaine, tout le monde repartira du même code. Pour que ce soit possible, les noms de classes et de fichiers sont fixés :

| classe                                         | fichier                                                   |
|------------------------------------------------|-----------------------------------------------------------|
| Personne                                       | personne.py                                               |
| Etudiant, TuteurEntreprise, EnseignantReferent | etudiant.py, tuteur_entreprise.py, enseignant_referent.py |
| Competence                                     | competence.py                                             |
| Offre, Candidature, Stage                      | offre.py, candidature.py, stage.py                        |

Les champs reprennent les mots de la responsable, sans accent : `nom`, `prenom`, `sexe`, `date_naissance`, `email`, `matricule`, `promotion`, `libelle`, `titre`, `description`, `date_debut`, `date_fin`, `nb_places`, `date_depot`, `statut`, `sujet`. Les noms des relations sont à votre choix.

Tout le reste — types, relations, contraintes, comportement à la suppression — est à décider.

## 2.2 Ce qu'elle ne dit pas

Son discours laisse des questions ouvertes. Tranchez-les, notamment :

- **Personne** doit-elle avoir sa propre table ? La responsable demande-t-elle jamais « toutes les personnes », sans distinction ?
- Le tuteur appartient-il à une entreprise, ou seulement au stage qu'il encadre ?
- Une candidature peut-elle donner lieu à deux stages ? Un stage peut-il exister sans candidature ? Quel champ l'interdit, et à quel niveau : Python, ou la base elle-même ?
- « Jamais deux fois à la même offre » : comment le garantir, et qui le vérifie ?
- La promotion : un nombre ou un texte ? Pensez à la statistique que l'IFNTI voudra un jour : le taux de placement par promotion.
- Le statut d'une candidature : du texte libre, ou une liste fermée ? Que coûte chaque choix le jour où une faute de frappe s'y glisse ?
- **Pour chaque relation, que se passe-t-il à la suppression ?** Relisez la dernière phrase de la responsable avant de répondre.

**Chaque arbitrage se défend dans un message de commit**, par une phrase que la responsable comprendrait. « PROTECT sur l'entreprise de l'offre » ne se comprend pas ; « on ne peut pas supprimer une entreprise qui a publié des offres, pour ne pas effacer leur historique » se comprend.

Faites plusieurs commits, dans un ordre qui raconte votre raisonnement : les personnes, puis les offres et les compétences, puis les candidatures et les stages.

## 3. Lire le SQL (20 min)

```
uv run manage.py makemigrations stages
uv run manage.py sqlmigrate stages 0002
uv run manage.py migrate
```

Si vous avez engendré plusieurs migrations au fil de vos commits, lisez-les toutes : le numéro de la commande du milieu change.

a. Combien de tables votre migration crée-t-elle ? Lesquelles ne correspondent à aucune classe que vous avez écrite, et pourquoi existent-elles ?

b. Y a-t-il une table pour **Personne** ? Est-ce cohérent avec votre choix de la partie 2 ?

c. Retrouvez dans le SQL la règle « jamais deux fois à la même offre ». Sous quelle forme apparaît-elle ?

d. Cherchez maintenant dans le SQL la trace de vos choix de suppression — **CASCADE**, **PROTECT**, ou ce que vous avez retenu. Que constatez-vous ? Qu'en déduisez-vous sur **qui** applique ces règles, et sur ce qui arriverait à une ligne supprimée directement en SQL, sans passer par Django ?

## 4. L'administration (10 min)

Déclarez tous vos modèles dans `stages/admin.py`. Pour chacun, choisissez les deux ou trois colonnes qui permettent à la responsable de s'y retrouver dans la liste — pas plus.

Puis, dans l'admin, essayez de supprimer une entreprise qui a publié au moins une offre.

a. Que se passe-t-il ? Est-ce ce que la responsable voulait ?

b. Si vous aviez choisi l'autre comportement, qu'aurait affiché l'admin, et qu'aurait-il supprimé ?

## 5. Peupler, puis interroger (35 min)

### 5.1 Un jeu de données cohérent

Depuis `uv run manage.py shell`, créez **au moins** : cinq compétences, trois entreprises dont deux à Sokodé, deux tuteurs, deux enseignants référents, cinq étudiants avec leurs compétences, trois offres avec les compétences qu'elles demandent, six candidatures de statuts variés, et deux stages issus de candidatures retenues.

Les données doivent être **cohérentes** : un stage naît d'une candidature retenue, et son tuteur travaille dans l'entreprise de l'offre. Rien, dans vos modèles, ne l'impose peut-être ; c'est à vous d'y veiller.

Gardez vos commandes dans un fichier `peuplement.py` à la racine du projet, pour pouvoir les rejouer sur une base neuve :

```
uv run manage.py shell < peuplement.py
```

### 5.2 Six requêtes

Écrivez dans le shell, puis recopiez dans un fichier `requetes.md`, la requête qui obtient :

1. les offres des entreprises situées à Sokodé ;
2. les étudiants qui possèdent la compétence « Django » ;
3. les candidatures d'un étudiant donné, **en partant de l'objet étudiant** ;
4. le nombre de candidatures retenues, sans charger les candidatures en mémoire ;
5. les stages dont l'offre vient d'une entreprise de Sokodé ;
6. les offres qui demandent au moins une compétence que possède un étudiant donné.

La sixième cache un piège. Comparez le nombre d'offres qu'elle renvoie au nombre d'offres qui existent vraiment. Si les deux ne concordent pas, trouvez pourquoi, et corrigez.

## 6. Ce que la base accepte (10 min)

Toujours dans le shell, et sans modifier vos modèles :

1. créez une offre qui **se termine avant de commencer** ;
2. créez une offre dont le titre est une chaîne vide ;
3. créez deux fois la même candidature, pour le même étudiant et la même offre.

a. Pour chacune des trois, que se passe-t-il ?

b. Deux d'entre elles relèvent du même problème. Lequel ? Qui, selon vous, aurait dû refuser ces données : la base, le modèle, ou autre chose ?

Notez vos réponses et **ne corrigez rien** : nous reviendrons sur ces offres dans quelques séances.

## À rendre

Poussez votre dépôt. L'historique doit contenir au moins un commit par étape de la partie 2, avec vos arbitrages dans les messages, ainsi que `peuplement.py` et `requetes.md`.

## Restitution (20 min)

Sans machine. Préparez de quoi répondre à deux d'entre elles.

1. Vous avez choisi un comportement à la suppression pour la relation entre un stage et sa candidature. Justifiez-le en une phrase que la responsable des stages comprendrait, puis dites ce qui se passerait si l'on avait choisi l'inverse.

---

---

---

---

2. La partie 3 vous a montré qui applique réellement vos règles de suppression. Donnez une situation concrète, dans la vie du projet, où cela pourrait laisser la base dans un état incohérent.

---

---

---

---

3. À la partie 6, la base a refusé une des trois créations et accepté les deux autres. Qu'avaient de différent les règles en jeu ?

---

---

---

---